

# **Agentic AI Prerequisites & Interview-Ready Guide**

A concise roadmap to design, build, and interview on agentic RAG systems

Prepared for: Bhavish  
Date: 2026-01-25

OVERVIEW Agentic AI (or agent-based systems) use an LLM as a controller that decides the control flow of an application. Rather than always running the same pipeline, an agent composes tools, retrieval, and LLM reasoning to decide what to do. Key idea: wrap existing RAG (retriever + LLM) as tools and add a Router Agent that decides whether to retrieve, summarize, or answer directly. This upgrade makes RAG dynamic, efficient, and more robust. Citation: LangChain blog 'What is an agent', LangChain Agents docs. [\[cite\turn0search8\turn0search18\]](#)

PREREQUISITES (SKILLS & KNOWLEDGE) - Machine Learning fundamentals: supervised learning, evaluation metrics, model selection. - NLP basics: tokenization, attention/transformer intuition, embeddings. - Probabilities & statistics: hypothesis testing, KS-test (for drift), A/B testing. - Software engineering: Python, package management, virtualenv, testing. - MLOps / LLMOps basics: containerization (Docker), CI/CD, model versioning, monitoring. - Tools & libraries to learn: LangChain (agents + tools), vector stores (Chroma/FAISS/Elasticsearch), transformer stacks (HF Transformers), and an LLM provider (OpenAI/Groq/HF/AWS Bedrock). Citations: LLMOps interview topics; RAG tutorials. [\[cite\]](#)[turn0search6](#)[turn0search12](#)

CORE CONCEPTS 1) Retrieval-Augmented Generation (RAG): Use a vector DB to fetch context relevant to a user query and feed that to the LLM to ground answers in source documents. This reduces hallucination and increases factuality. 2) Tools & Chains: Tools are functions (e.g., 'search\_documents', 'summarize') that agents can call. Chains are predefined pipelines that wire LLM calls and tool calls. 3) Agent Roles: - Router Agent: decides if retrieval is needed or direct LLM answer is enough. - Retrieval Agent (RAG tool): performs vector search, returns top-k chunks. - Answer Agent: generates the final response, formats output, and cites sources if needed. Citation: LangChain RAG + LangGraph agentic RAG guide. [\[cite\]](#)[\[turn0search12\]](#)[\[turn0search3\]](#)

PRACTICAL DESIGN CHECKLIST Architecture components to implement: - Ingestion: PDF/text loader, chunking, embeddings pipeline. - Vector DB: Chroma/FAISS/Elasticsearch with metadata support. - RAG tool: Retriever wrapper returning top-k chunks + metadata. - Agents: Router Agent (rule-based or LLM-based), Retrieval Agent (tool), Answer Agent. - Model/Tooling: LLM provider (Gemma-2b / OpenAI / Anthropic), LangChain agents or custom tool wrappers. - Observability: logging for queries, drift detection for retriever inputs, metrics for answer quality. Design notes: - Start with a rule-based Router; upgrade to LLM-based routing when stable. - Persist feature/schema and vector versions; version your index when reindexing. - Keep tools idempotent and testable; create unit tests for each tool.

IMPLEMENTATION ROADMAP (STEP-BY-STEP) Phase 1: Stabilize RAG - Ensure ingestion + chunking works, store embeddings, and verify retrieval quality. - Implement evaluation: retrieval recall, answer quality, and human-in-the-loop checks. Phase 2: Add Router Agent (rule-based) - Implement a lightweight router using heuristics (keywords, question type, length). - Expose RAG as a tool (search documents) and an LLM-only tool. Phase 3: Add Answer Agent - Use retrieved context + chain to generate final answers and citations. - Add prompt templates and safety constraints. Phase 4: Upgrade Router to LLM-based - Provide the LLM with a short system prompt + examples to decide which tool to call. - Add guardrails and fallbacks to the rule-based router. Phase 5: Optional action tools - Add safe action tools if needed (open URLs, return links, call APIs). Ensure security and authorization.

INTERVIEW PREP — TOPICS & SAMPLE QUESTIONS Focus areas: - RAG fundamentals & trade-offs (why retrieval helps, chunk sizes, context windows). - LangChain agents & tools (how to design a tool; safety and tool authorization). - LLMops: model versioning, prompt engineering, monitoring, cost optimization. - Evaluation: PR-AUC, online vs offline eval, human evaluation, hallucination reduction. Sample questions: 1. Explain how RAG reduces hallucination and what retrieval mistakes can still cause hallucination. 2. How would you design a Router Agent? What features or signals would you use? 3. How do you version vector indexes and handle reindexing in production? 4. Describe safety considerations for adding action tools (e.g., open web pages, buy items). Citations: LLMops resources and interview guides. [\[cite\]](#)[turn0search6](#)[\[turn0search16\]](#)

RESOURCES — READING & TUTORIALS - What is an AI agent? — LangChain blog. [\[cite\turn0search8\]](#) - LangChain Agents docs — official. [\[cite\turn0search18\]](#) - RAG tutorials (LangChain & AWS). [\[cite\turn0search12\turn0search9\]](#) - LangGraph Agentic RAG guide — advanced patterns. [\[cite\turn0search3\]](#) - LLM/Ops interview checklist & questions. [\[cite\turn0search6\turn0search16\]](#) Use these for deep dives and follow their code samples for hands-on practice.



PRACTICE PROJECTS (BUILD THESE) 1) Agentic RAG Q&A: build router (rule-based), retrieval tool (Chroma), answer agent (LangChain chain). Document decisions. 2) Multi-step comparison agent: given two papers, return a comparison table of methods, pros/cons, and citations. 3) RAG + Action toy: after answer, produce a 'Related Links' widget with safe URLs — no web automation yet. Each project should have tests, a README, and a short demo video.